



Qiskit Architecture

Structure, Transpilation Pipeline, and Native Gate Passthrough to IonQ

QUANTUM ADVANTAGE LAB — SDK REFERENCE

1. Qiskit Stack Overview

Qiskit is an open-source quantum SDK maintained by IBM. Its modular, layered architecture separates what you want to compute from how it runs on hardware. For Quantum Advantage Lab, we use Qiskit as the circuit construction and compilation layer, paired with the **Qiskit IonQ Provider** to target IonQ hardware specifically.

USER LAYER

QuantumCircuit API — Build circuits with high-level gates (H, CNOT, Rz, custom unitaries). Algorithm libraries (Grover, VQE, Hamiltonian simulation).

TERRA CORE

Circuit IR + DAG — Internal DAG representation, circuit optimization passes, parameterized circuits, Pauli operator algebra.

TRANSPILER

Compilation Pipeline — Maps abstract gates → target basis gates. Routing, optimization levels 0-3, custom pass managers. This is where native gate control happens.

PROVIDER

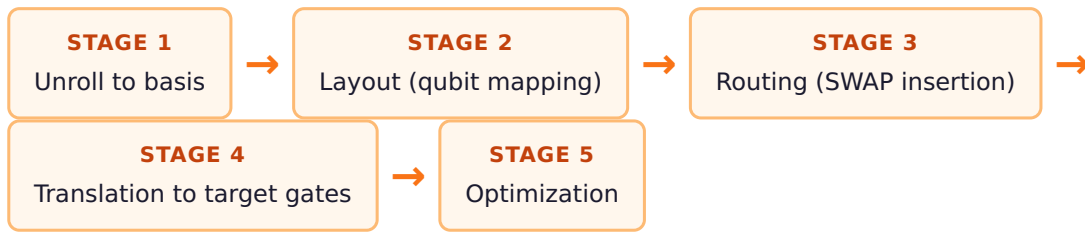
Qiskit IonQ Provider — Translates compiled circuits to IonQ API format. Handles job submission, status polling, result retrieval. Supports native gate passthrough.

HARDWARE

IonQ QPU — Aria / Forte trapped-ion systems. Executes {GPI, GPI2, MS} gate set on physical ions.

2. The Transpilation Pipeline

Qiskit's transpiler converts user-written circuits into hardware-executable form. It runs as a sequence of **passes** organized into stages:



For IonQ, Stage 3 (routing) is trivial — all-to-all connectivity means no SWAP gates are ever inserted. The transpiler's routing pass becomes a no-op, and the circuit's logical depth equals its physical depth. This is a major structural advantage baked into the compilation pipeline.

3. Native Gate Passthrough to IonQ

By default, Qiskit transpiles to an abstract basis (e.g., {CX, Rz, SX}) and the IonQ provider re-compiles to native gates server-side. This double-transpilation introduces unnecessary overhead. **Native gate passthrough** lets us compile directly to IonQ's {GPI, GPI2, MS} in Qiskit, sending already-native circuits that IonQ executes as-is.

Step 1: Define Native Gate Classes

```
from qiskit.circuit import Gate
import numpy as np

class GPIGate(Gate):
    """IonQ native GPI gate:  $\pi$  rotation about axis at angle  $\phi$  in XY-plane."""
    def __init__(self, phi):
        super().__init__('gpi', 1, [phi])

class GPI2Gate(Gate):
    """IonQ native GPI2 gate:  $\pi/2$  rotation about axis at angle  $\phi$ ."""
    def __init__(self, phi):
        super().__init__('gpi2', 1, [phi])

class MSGate(Gate):
    """IonQ native MS (Mølmer-Sørensen) gate: XX-type entangling gate."""
    def __init__(self, phi0, phi1, angle=0.25):
        super().__init__('ms', 2, [phi0, phi1, angle])
```

Step 2: Build Circuits with Native Gates Directly

```
from qiskit import QuantumCircuit

qc = QuantumCircuit(3)

# Hadamard equivalent: GPI2( $\pi/4$ ) then GPI(0)
qc.append(GPI2Gate(np.pi/4), [0])
qc.append(GPIGate(0), [0])

# Entangle qubits 0 and 2 directly (all-to-all, no SWAP needed)
qc.append(MSGate(0, 0), [0, 2])

# Single-qubit rotation (virtual-Z via phase tracking)
qc.append(GPI2Gate(np.pi/3), [1])
```

Step 3: Submit with Native Gate Passthrough

```
from qiskit_ionq import IonQProvider

provider = IonQProvider("YOUR_API_KEY")
backend = provider.get_backend("ionq_aria")

# Key: gateset="native" tells IonQ to execute gates as-is
job = backend.run(qc, gateset="native", shots=1024)
result = job.result()
```

The `gateset="native"` flag is the critical setting. Without it, IonQ's server re-transpiles your circuit — potentially undoing your careful native compilation. With it, the circuit executes exactly as submitted: your GPI, GPI2, and MS gates map 1:1 to physical laser pulses.

4. Custom Transpiler Pass for IonQ Native Compilation

For our project, we build a custom Qiskit transpiler pass that compiles abstract circuits to IonQ native gates locally, giving us full control over optimization:

```
from qiskit.transpiler import PassManager
from qiskit.transpiler.passes import (
    Unroller, Optimize1qGatesDecomposition, CXCancellation
)

def ionq_native_pass_manager(optimization_level=2):
    """Custom pass manager: abstract gates → IonQ native {GPi, GPi2, MS}."""

    pm = PassManager()

    # Stage 1: Decompose to {CX, U3} intermediate basis
    pm.append(Unroller(['cx', 'u3']))

    # Stage 2: Cancel adjacent CX pairs
    pm.append(CXCancellation())

    # Stage 3: Rewrite CX → MS + single-qubit wrappers
    pm.append(CXToMSPass())          # custom: CX → GPi2 + MS + GPi2

    # Stage 4: Rewrite U3 → GPi + GPi2 sequences
    pm.append(U3ToNativePass())      # custom: U3(θ, φ, λ) → GPi/GPi2 chain

    # Stage 5: Merge adjacent single-qubit gates
    pm.append(Optimize1qGatesDecomposition(
        basis=['gpi', 'gpi2']
    ))

    return pm

# Usage
pm = ionq_native_pass_manager()
native_circuit = pm.run(abstract_circuit)
```

5. Integration Architecture for Quantum Advantage Lab



Per-Module Circuit Construction

MODULE	QISKIT BUILDING BLOCK	NATIVE GATE STRATEGY	KEY OPTIMIZATION
Grover's	<code>GroverOperator</code> + custom oracle	Oracle multi-CZ → parallel MS gates	All-to-all: oracle = single MS layer
Quantum Walk	Custom coin + shift unitaries	Shift permutations → MS + GPI chains	No SWAP routing for dense graphs
VQE	<code>EfficientSU2</code> ansatz + <code>SparsePauliOp</code>	Entangling layers = direct MS gates on all pairs	Full-connectivity ansatz, fewer layers needed
Simulation	<code>PauliEvolutionGate</code> + Trotter formula	XX terms → single MS gate each	MS is natively XX; ~3× fewer gates than CX decomposition

6. Concrete Example: VQE on LiH

End-to-end flow showing how Qiskit constructs, transpiles, and submits a VQE circuit for lithium hydride to IonQ Aria:

```
from qiskit.circuit.library import EfficientSU2
from qiskit.quantum_info import SparsePauliOp
from qiskit.primitives import Estimator
from scipy.optimize import minimize

# 1. Define the LiH Hamiltonian (10-qubit, STO-3G)
hamiltonian = SparsePauliOp.from_list([
    ("IIIIIIIZZ", -0.8105),
    ("IIIIIIIZZ", 0.1721),
    # ... ~100 Pauli terms for LiH
])

# 2. Hardware-efficient ansatz: FULL entanglement (IonQ all-to-all)
ansatz = EfficientSU2(
    num_qubits=10,
    entanglement="full",    # ← every qubit pair entangled each layer
    reps=2,                 # ← only 2 layers needed (full connectivity)
    su2_gates=['ry', 'rz']
)

# 3. Transpile to IonQ native gates
pm = ionq_native_pass_manager(optimization_level=2)
native_ansatz = pm.run(ansatz)

# 4. VQE cost function
estimator = Estimator(backend=backend, options={"gateset": "native"})

def cost_fn(params):
    bound = native_ansatz.assign_parameters(params)
    energy = estimator.run(bound, hamiltonian).result().values[0]
    # Stream intermediate energy to frontend via WebSocket
    ws.send_json({"step": iteration, "energy": energy})
    return energy

# 5. Run optimizer
result = minimize(cost_fn, x0=initial_params, method="COBYLA")
```

Why `entanglement="full"` Matters

On a grid-topology device, `entanglement="full"` would require $O(n)$ SWAP layers per ansatz layer — making it impractical beyond ~ 8 qubits. On IonQ, every pair connects directly via a single MS gate. This means:

- **2 layers on IonQ** achieves the same expressivity as **6-8 layers on a grid device**
- Fewer layers = fewer parameters = faster classical optimization convergence
- Fewer gates = less error accumulation = cleaner energy estimates
- The VQE "race" visualization shows genuine convergence, not noise

7. Simulator Fallback (Aer)

For practice mode and development, Qiskit's **Aer simulator** provides local execution:

```
from qiskit_aer import AerSimulator

# Ideal simulation (no noise)
sim = AerSimulator(method="statevector")

# Noise model mimicking IonQ Aria error rates
from qiskit_aer.noise import NoiseModel, depolarizing_error
noise = NoiseModel()
noise.add_all_qubit_quantum_error(depolarizing_error(0.003, 2), ['ms'])
noise.add_all_qubit_quantum_error(depolarizing_error(0.0003, 1), ['gpi', 'gpi2'])
sim_noisy = AerSimulator(noise_model=noise)
```

Three execution modes in the platform:

1. **Ideal simulator** — instant results, perfect fidelity (teaching baseline)
 2. **Noisy simulator** — IonQ-calibrated noise model (realistic preview)
 3. **IonQ hardware** — real QPU execution (the real race)
- Users can compare all three, learning about noise and error mitigation hands-on.